

Test 1 Algorithms

$$\begin{array}{r} 76 \\ +1 \\ \hline 77 \end{array}$$

Name Brandon Buchanan

2.23.12

1) (4 points) Complete the following table for $n=16$, find $f(16)$ for each function

Function	$n = 16$
$f(n)=$	
$\lg n$	4
n	16
$n \lg n$	64
n^2	256
n^3	65535
2^n	6144
$n!$	16!

6×1024

2 a) (4 points) Suppose you have a machine that runs 1 instruction per second. Determine the largest input size that can be processed for each function within 64 seconds.

function	32 seconds	64
$\lg n$	2048	
n	64	
$n \lg n$	16	
n^2	8	
n^3	4	
2^n	8	
$n!$	4	$4 < n < 5$

b) (4 points) For the 64 seconds table above, what coefficient could we multiply n^2 by so that it would process less input than $n!$

$$> 4$$

3) (10 points) List $T(2)$, $T(3)$ and $T(4)$ and then Solve the following recurrence relation by substitution

a) $T(n) = T(n-1) + 1$ and $T(1) = 1$

b) $T(n) = 2T(n-1) + 1$ and $T(1) = 1$

a) $T(n-1) = T(n-2) + 1$ $T(n) = T(n-1) + 1 = T(n-2) + 1 + 1$
 $T(n-2) = T(n-3) + 1$ $T(n) = T(n-2) + 1 + 1 = T(n-3) + 1 + 1 + 1$

$T(1) = 1 = n - n + 1$

$T(2) = n - (n-1) + 1 = 2$

$T(3) = n - (n-2) + 1 = 3$

$T(4) = n - (n-3) + 1 = 4$ ✓

$T(n) = (n - (n-1)) + 1$ -3

b) $T(n-1) = 2T(n-2) + 1$ $T(n) = 2T(n-1) + 1 = 2(2T(n-2) + 1) + 1$
 $T(n-2) = 2T(n-3) + 1$ $T(n) = 4T(n-2) + 2 \cdot 1 + 1$
 $4(2T(n-3) + 1) + 2 \cdot 1 + 1$
 $8T(n-3) + 4 \cdot 1 + 2 \cdot 1 + 1$

$T(2) = 2(1) + 1 = 3$

$T(3) = 3(3) + 1 = 10$

$T(4) = 4(10) + 1 = 41$

$2^{n-1} + \sum_{i=0}^{n-1} 2^i = 2^{n-1} - 1$ -1

4) (10 Points) Solve

a) $\sum_{i=3}^n 2i = \sum_{i=1}^n 2i - \sum_{i=1}^2 2i = n(2i) - 4i = 2i(n-2)$ -6

b) $\sum_{i=1}^n 4^i$ ~~$n 4^i$~~

-10

5) (16 points) Solve the following recurrence relation by Master's Method

a) $T(n) = 8T(n/2) + n^2$ $a=8, b=2, f(n)=n^2$
 $n^2 = O(n^{3-\epsilon}), \epsilon > 0, \text{ true}$

$$\Theta(n^3) = T(n)$$

b) $T(n) = 8T(n/2) + n$ $a=8, b=2, f(n)=n$
 $n = O(n^{3-\epsilon}), \text{ true}$

$$\Theta(n^3) = T(n)$$

c) $T(n) = 8T(n/2) + n^3$ $a=8, b=2, f(n)=n^3$
 $n^3 = O(n^{3-\epsilon}), \text{ false}$
 $n^3 = \Theta(n^3), \text{ true}$

$$\Theta(n^3 \lg n) = T(n)$$

d) $T(n) = 8T(n/2) + n^4$

$$n^4 = O(n^{3-\epsilon}), \text{ false}$$

$$n^4 = \Theta(n^3), \text{ false}$$

$$n^4 = \Omega(n^{3+\epsilon}), \text{ true}$$

$$8\left(\frac{n^4}{2}\right) \leq 0.5(n^4)$$

$$\Theta(n^4) = T(n)$$

- 6) (6 points) Find $T(n)$ for the following recursive function and solve using Master's Method.
 // A is an array of items

```

Max (A, first, last)
{
  if first > last return -∞ //array is empty
  if first == last return A[first]

  return
  Maximum (
    Max (A, first, (first+last)/3),
    Max (A, (first+last)/3+1, 2*(first+last)/3),
    Max (A, 2*(first+last)/3+1, last)
  )
}

```

$$T(n) = 3T(n/3) + \cancel{X}^4 \quad a=3, b=3, f(n) = \log_3 n$$

$$\log_3 n = O(n^{1-\epsilon}), \quad \epsilon > 0, \quad \text{true}$$

$$T(n) = O(n)$$

(-2)

7) (10 Points)

a) Write code for binary search which breaks the sorted list in three parts, i.e. look at two numbers (one at 1/3 through the list and one at 2/3 through the list). Then based on those two numbers looks at the 1/3 of the list recursively.

b) Find the $T(n)$ and solve using Masters Method

a) `TrimySearch(list, start, first, second, end, key)`
 {

```

  if (first == second)
    return list[first]
  if (first < second)
    return null

```

```

  if (first > key)
    return TrimySearch(list, start, first, first*2/3, first, key)

```

```

  if (first < key && second > key)

```

```

    return TrimySearch(list, first, (first+second)/3, 2*(first+second)/3, second, key)

```

```

  if (second < key)

```

```

    return TrimySearch(list, second, (second+end)/3, 2*(second+end)/3, key)

```

}

b) $T(n) = O(n)$

(4)

8) (8 points) Each of the following calculates $p(x)$, where

$$p(x) = a_0 + a_1x + a_2x^2 + a_3x^3 + \dots + a_{n-1}x^{n-1} + a_nx^n$$

a) double Evaluate_P (double x)

// x the value of the input

Result = a[0]+a[1]*x

Xpower=x

for I = 2 to n

xPower=xPower*x

result=result +a[i]*xPower

end for

return result

- How many multiplications? $2n-1$ ✓
- How many additions? $2n-1$ ✓
- How many total operations? $4n-2$
- What is the running time in terms of Big Oh? $O(n)$

b) Horner's Method is a slight faster approach

double HonersMethod_Evaluate_P (double x)

// x the value of the input

Result = a[n]

for I = n-2 down to 0

result=result*x +a[i]

end for

return result

- How many multiplications? $n-1$ ✓
- How many additions? $n-1$
- How many total operations? $2n-2$
- What is the running time in terms of Big Oh? $O(n)$

9) (5 points) Use the following data set to carry out each sort:

index	1	2	3	4
value	21	115	120	107

Bubble sort

pass 1: 21, 115, 107, 120
pass 2: 21, 107, 115, 120
pass 3: 21, 107, 115, 120

Merge sort

[21, 115] [120, 107] split the list

[21] [115] [120] [107] split both lists again, then recombine

[21, 115] [107, 120] two lists, each are sorted

[21, 107, 115, 120] merge both lists



10) (15 points) Which of the following functions are in

- a) $\Omega(n)$ $F_2, F_3, F_6, F_7, F_8, F_9, F_{10}$ F_4 (-2)
 b) $O(n^2)$ $F_1, F_3, F_4, F_5, F_6, F_{10}$ F_7 (-2)
 c) $\Theta(\lg n)$ F_1 F_5

$$F_1(n) = \lg n$$

$$F_2(n) = 2^n$$

$$F_3(n) = (\lg n)^2$$

$$F_4(n) = \sqrt{n}$$

$$F_5(n) = \lg(n^2) = n$$

$$F_6(n) = 3^n$$

$$F_7(n) = n^2$$

$$F_8(n) = n^2(\lg n)$$

$$F_9(n) = n!$$

$$F_{10}(n) = 7$$

d) Explain why $2^n = \Theta(2^{n-1})$

The running time is within the same class because both n and $n-1$ are considered to be in the same class of functions; likewise, 2^n & 2^{n-1} are also in the same class of functions.

11) (8 points) For multiplying two integers m and n , the following naive code runs in $O(m)$

↑ this code doesn't know what's up

Multiply (m, n)

result = 0

for $I = 1$ to m

result = result + n

end for

return result

- a) How could we make a simple modification of the code to improve the running time to $O(\text{minimum}\{m, n\})$? Reduce the number of additions by utilizing previously calculated results - $m(\frac{n}{2}) + m(\frac{n}{2})$
 b) Write a *Multiply* that is faster (in complexity, i.e. big O , than part a)

Multiply (m, n)

if ($n == 1$)

return n

else

return $\text{Multiply}(m, n/2) + \text{Multiply}(m, n/2)$

$$T(n) = 2T(\frac{n}{2}) + 1$$

$$O(n \log n)$$

↑ (-1)